CULT Asset Manager

Design Document — IIT Roorkee Cultural Council

Project	Smart Asset Management & Resource Allocation Platform
Team	IIT Roorkee — Cultural Council
Live App	https://asset-manager-iitr.vercel.app
GitHub	https://github.com/Ronak-F7/Asset-Manager-IITR

1. Problem Understanding

The Cultural Council of IIT Roorkee manages a large pool of shared equipment -- cameras, audio systems, lighting, costumes, and event infrastructure — across multiple student sections and events.

Until now, managing these assets meant a lot of back and forth messages, shared spreadsheets, and handwritten registers. This made it hard to track what's available, who has what, and when things are due back.

The core problems I set out to solve:

- No central place to see what's available right now
- Booking conflicts — two people requesting the same item
- No formal approval process — things were taken without proper sign-off
- No record of who had what and when it was returned
- Zero visibility for coordinators into overall usage patterns

2. System Architecture

The app is split into three layers that work together.

Layer	Technology	What it does
Frontend	React + Vite + Tailwind CSS	The UI — what users see and interact with
Backend	Node.js + Express	Handles all the logic — logins, bookings, approvals
Database	PostgreSQL (Neon)	Stores all data — users, assets, bookings, logs

Auth	JWT Tokens	Keeps sessions secure without storing passwords plainl
Charts	Recharts	Draws the dashboard graphs and analytics

y

How a request flows through the system:

User opens the app (Frontend) → clicks Book Asset → Frontend sends a request to the Backend → Backend checks the database to confirm availability → saves the booking → sends confirmation back → user sees success message.

3. Database Scheme

We have 5 tables. Each table stores one type of information, and they reference each other using IDs.

Users

Stores everyone who can log in.

- id — unique identifier
- name, email — basic info
- password — stored as a secure hash (never plain text)
- role — either ADMIN (For this project I just made 1 admin) or USER

Assets

Every piece of equipment the council owns.

- id, name, category, description
- totalQuantity — how many exist in total
- availableQuantity — how many are free right now
- status — Available /Partially Avaialable / Unavailaible / Maintenace
- condition — Excellent / Good / Fair / Poor
- qrCode — auto-generated QR image for physical tagging

Bookings

Every request made by a user.

- userId, assetId — who requested what
- quantity, startDate, endDate, purpose
- status — Pending→ Approved→ Issued → Returned
- adminNote — note left by admin when approving/rejecting
- issuedAt, returnedAt — exact timestamps

AuditLogs

A record of every important action taken.

- action — e.g. BOOKING_APPROVED, ASSET_CREATED
- userId, assetId — who did what to which asset

· details — human-readable description

MaintenanceLogs

Tracks damage reports and repairs.

· assetId — which asset
· description — what happened
· reportedBy, resolvedAt

4. Entity Relationship Diagram (ERD)

Entity	Relates to	Relationship
User	Booking	One user can have many bookings
Asset	Booking	One asset can appear in many bookings
Booking	User + Asset	Each booking belongs to one user and one asset
Asset	MaintenanceLog	One asset can have many maintenance records
User	AuditLog	One user can trigger many audit log entries
Asset	AuditLog	One asset can appear in many audit log entries

5. API Overview

The backend exposes a REST API — a set of URLs the frontend calls to get or save data. Here are the main ones:

Method	Endpoint	What it does
POST	/api/auth/register	Create a new account
POST	/api/auth/login	Login and get a session token
GET	/api/assets	List all assets (with search & filter)
POST	/api/assets	Add a new asset (admin only)
PUT	/api/assets/:id	Edit an asset (admin only)
DELETE	/api/assets/:id	Delete an asset (admin only)
POST	/api/bookings	Submit a booking request
PATCH	/api/bookings/:id/approve	Approve a request (admin only)
PATCH	/api/bookings/:id/reject	Reject a request (admin only)
PATCH	/api/bookings/:id/issue	Mark asset as issued (admin only)
PATCH	/api/bookings/:id/return	Mark asset as returned (admin only)

GET	/api/analytics/dashboard	Get dashboard stats and chart data
GET	/api/analytics/utilization	Get per-asset utilization rates
GET	/api/audit	Get full audit log (admin only)

6. Design Decisions

Role-based access

We have two roles — Admin and User. Admins can manage everything. Users can only browse and request. This keeps the system safe without being complicated.

JWT Authentication

When you log in, you get a token. Every request carries this token so the server knows who you are without asking for your password again.

Inventory auto-update

When a booking is approved and issued, the available quantity drops automatically. When returned, it goes back up. No manual counting needed.

QR Codes

Every asset gets a unique QR code when created. Print and stick it on the equipment so admins can scan it during issue/return instead of searching by name.

Audit Logs

Every important action — creating an asset, approving a booking, returning equipment — gets logged with who did it and when. Full accountability.

Utilization Analytics

Admins can see which assets are heavily used vs barely touched. Helps make smarter decisions — buy more of what's always booked, retire what's never used.

Free cloud deployment

We used Vercel (frontend), Render (backend), and Neon (database) — all free tiers. Live 24/7 with no server management needed.

Ronak Das

Enroll no : 24117043

B.Tech Mechanical Dept